Nodal DG method for the 1D advection equation

This text was mainly inspired by [1]. My own code supporting this text can be found on Github.

Contents

Symbols and definitions

- Ω - our domain
- u_h - approximate solution obtained on a mesh with spatial step h
- $(u, v)_{L^2(\Omega)} = \int_\Omega uv \, \mathrm{d}x$ - scalar product in L^2
- $\| \ u \ \|^2_{L^2(\Omega)} = (u, u)_{L^2(\Omega)}$
- $P_j\left(D^k\right)$ - space of polynomoials of up to order j on an interval D^k
- φ_i, ψ_i - i-th basis function
- u^- - limit of u when approaching the boundary of an element from the left
- u^+ - limit of u when approaching the boundary of an element from the right
- $f^* = f^*(u^-, u^+)$ - numerical flux
- $\hat{\boldsymbol{n}}$ - outward unit normal
- $\hat{\boldsymbol{n}}^-$ - outward unit normal on the left face of an element (-1 in 1D)
- $\hat{\boldsymbol{n}}^+$ - outward unit normal on the right face of an element (1 in 1D)
- $\{\!\!\{u\}\!\!\} = \frac{u^- + u^+}{2}$
- $[\![u]\!] = \hat{\boldsymbol{n}}^- u^- + \hat{\boldsymbol{n}}^+ u^+$
- $[\![\boldsymbol{u}]\!] = \hat{\boldsymbol{n}}^- \cdot \boldsymbol{u}^- + \hat{\boldsymbol{n}}^+ \cdot \boldsymbol{u}^+$
- $l_i(x)$ - i-th Lagrange polynomoial
- N - order of polynomial approximation
- $N_p = N + 1$ - number of points in each element (number of degrees of freedom)

1. Problem overview

The general one dimensional advection equation is of the following form:

$$\frac{\partial \boldsymbol{u}}{\partial t} + \frac{\partial \boldsymbol{f}(\boldsymbol{u})}{\partial x} = \boldsymbol{s}(x,t) \quad [x,t] \in \Omega \times \mathbb{R}^+$$

where $\Omega = (L, R)$ is our domain, $\boldsymbol{f} = [f_1(\boldsymbol{u}), f_2(\boldsymbol{u}), ..., f_n(\boldsymbol{u})]^T$ is the physical flux, $\boldsymbol{u} = \boldsymbol{u}(x,t) = [u_1, u_2, ..., u_n]^T$ is the unknown vector valued function and $\boldsymbol{s}(x,t) = [s_1, s_2, ..., s_n]$ is the source term. This equation is hyperbolic.

For this problem to be well defined we need to impose initial and boundary conditions.

$$\begin{aligned} \boldsymbol{u}(x,0) &= \boldsymbol{u}_0(x), \\ \mathcal{B}_L \boldsymbol{u}(L,t) &= \boldsymbol{g}_1(t) \quad \text{at} \quad x = L, \\ \mathcal{B}_R \boldsymbol{u}(R,t) &= \boldsymbol{g}_2(t) \quad \text{at} \quad x = R, \end{aligned}$$

where the sum of the ranks of the boundary operators $\mathcal{B}_L, \mathcal{B}_R$ equals the number of the required inflow conditions.

1.1. Weak formulation of the problem

To obtain the weak formulation we multiply the equation by a test function $\boldsymbol{v} \in \mathcal{V} = \mathcal{C}_0^\infty(\Omega)$ and integrate over the domain Ω.

$$\int_\Omega \boldsymbol{v}^T \frac{\partial \boldsymbol{u}}{\partial t} \,\mathrm{d}x + \int_\Omega \boldsymbol{v}^T \frac{\partial \boldsymbol{f}(\boldsymbol{u})}{\partial x} \,\mathrm{d}x = \int_\Omega \boldsymbol{v}^T \boldsymbol{s}(x,t) \,\mathrm{d}x.$$

Using per-partes on the second integral we obtain

$$\int_\Omega \boldsymbol{v}^T \frac{\partial \boldsymbol{u}}{\partial t} \,\mathrm{d}x - \int_\Omega \frac{\partial \boldsymbol{v}^T}{\partial x} \boldsymbol{f}(\boldsymbol{u}) \,\mathrm{d}x + \int_{\partial\Omega} \hat{\boldsymbol{n}} \cdot \boldsymbol{v}^T \boldsymbol{f}(\boldsymbol{u}) \,\mathrm{d}x = \int_\Omega \boldsymbol{v}^T \boldsymbol{s}(x,t) \,\mathrm{d}x,$$

where $\hat{\boldsymbol{n}}$ is the outward unit normal on the domain boundary $\partial\Omega$. Notice that we express the term $\left[\boldsymbol{v}^T \boldsymbol{f}(\boldsymbol{u})\right]_L^R$ in the integral form $\int_{\partial\Omega} \hat{\boldsymbol{n}} \cdot \boldsymbol{v}^T \boldsymbol{f}(\boldsymbol{u}) \,\mathrm{d}x$, this will later help us when moving to multiple dimensions.

2. Discontinous Galerkin Method

Let's focus on linear scalar advection equation without source term for now and generalize the scheme later.

2.1. Spatial discretization

We split our domain $\Omega = (L, R)$ into N elements

$$D^k = \langle x_{k-\frac{1}{2}}, x_{k+\frac{1}{2}} \rangle, \; k = 1, 2, ..., K,$$

here x_k is the center of the k-th element and $x_{\frac{1}{2}} = L$, $x_{N+\frac{1}{2}} = R$.

$$\bar{\Omega} \approx \bar{\Omega}_h = \bigcup_{k=1}^{K} D^k$$

We now formulate the local weak formulation

$$\int_{D^k} v \frac{\partial u}{\partial t} \,\mathrm{d}x = \int_{D^k} \frac{\partial v}{\partial x} f(u) \,\mathrm{d}x - \int_{\partial D^k} \hat{\boldsymbol{n}} v f(u) \,\mathrm{d}x$$

But now we encounter a problem, because u is double valued at the element boundaries. What vallue should we choose when evaluating the $\int_{\partial D^k} \hat{\boldsymbol{n}} v f(u)\,\mathrm{d}x$ term? To solve this we define a numerical flux $f^* = f^*(u^-, u^+)$ that takes in both values and returns only one. The equation then becomes

$$\int_{D^k} v\frac{\partial u}{\partial t}\,\mathrm{d}x = \int_{D^k} \frac{\partial v}{\partial x} f(u)\,\mathrm{d}x - \int_{\partial D^k} \hat{\boldsymbol{n}} v f^*(u^-, u^+)\,\mathrm{d}x$$

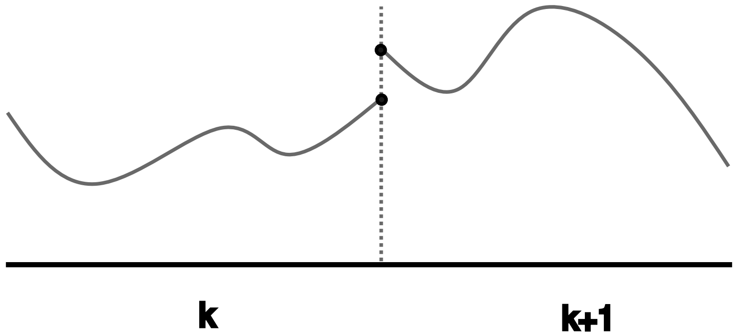


Figure 1: u_h is multivalued at the elemental interfaces

Lax-Friedrichs flux

One such numerical flux is the local Lax-Friedrichs numerical flux.

$$f^*(u^-, u^+) = \frac{f(u^-) + f(u^+)}{2} + \frac{C}{2}\hat{\boldsymbol{n}}(u^- - u^+)$$

where the local constant C is determined by the maximum local advection speed

$$C = \max_{u^- \leq s \leq u^+} f_u(s)$$

and $\hat{\boldsymbol{n}}$ is local unit outward normal. In the 1D example it becomes -1 on the left face of an element and 1 on the right face.

Upwind flux

$$f^*(u^-, u^+) = \begin{cases} f(u^-) & : \quad \hat{\boldsymbol{n}}C > 0 \\ f(u^+) & : \quad \hat{\boldsymbol{n}}C < 0 \end{cases}$$

where $C = f(u^-)$ i.e. the local advection speed.

Godunov flux

$$f^*(u^-, u^+) = \begin{cases} \min(f(u^-), f(u^+)) & : \quad \hat{\boldsymbol{n}}u^- < \hat{\boldsymbol{n}}u^+ \\ \max(f(u^-), f(u^+)) & : \quad \hat{\boldsymbol{n}}u^- \geq \hat{\boldsymbol{n}}u^+ \end{cases}$$

2.2. Basis functions

We assume that the approximate solution $\boldsymbol{u}_h(x,t)$ can be expressed as a direct sum of local piecewise polynomial solutions

$$u(x,t) \approx u_h(x,t) = \bigoplus_{k=1}^{K} u_h^k(x^k, t).$$

We define a local function space V_h^k such that

$$V_h = \bigoplus_{k=1}^{K} V_h^k$$

$$V_h^k = \left\{\varphi \in L^2\left(D^k\right) : \varphi\mid_{D^k} \in P_j\left(D^k\right),\, j = 1, 2, ..., N\right\},$$

where $P_j\left(D^k\right)$ is a space of polynomials of at most degree j on the interval D^k.

Now we can express the local approximate solution in the basis of the space V_h^k

$$x \in D^k \quad : \quad u_h^k(x,t) = \sum_{n=1}^{N_p} \hat{u}_n^k(t)\varphi_n(x) = \sum_{i=1}^{N_p} u_h^k(x_i,t) l_i(x),$$

where $\hat{u}_n$ is a vector of coefficients, φ_n is n-th basis function and l_i is the i-th interpolating Lagrange polynomial, we assume that $x_i \in D^k$ are distinct. The first expression is said to be modal representation and the second nodal. We won't discuss the modal formulation any further in this text, but it will come in handy when implementing limiters later when formulating the scheme for general non-linear problems.

Following the Galerkin approach we replace the test function with each of the basis functions. This yields a system of $N+1$ equations for each element

$$\int_{D^k} l_i(x)\frac{\partial}{\partial t}\left(\sum_{j=1}^{N_p} u_h\left(x_j,t\right)\right)\mathrm{d}x = \int_{D^k}\frac{\mathrm{d}l_i(x)}{\mathrm{d}x}\sum_{j=1}^{N_p} a u_h\left(x_j,t\right) l_j(x)\,\mathrm{d}x - \int_{\partial D^k} l_i f^*\,\mathrm{d}x$$

$$\sum_{j=1}^{N_p}\frac{\mathrm{d}}{\mathrm{d}t}u_h\left(x_j,t\right)\int_{D^k} l_i(x)l_j(x)\,\mathrm{d}x = \sum_{j=1}^{N_p} a u_h\left(x_j,t\right)\int_{D^k}\frac{\mathrm{d}l_i(x)}{\mathrm{d}x}l_j(x)\,\mathrm{d}x - \int_{\partial D^k} l_i f^*\,\mathrm{d}x$$

$$i = 1, 2, ..., N+1$$

We can rewrite this equation into it's matrix form

$$M^k\frac{\mathrm{d}\boldsymbol{u}_h^k}{\mathrm{d}t} = a\left(S^k\right)^T\boldsymbol{u}_h^k - \mathcal{E}\boldsymbol{f}_*^k,$$

where

$M_{ij}^k = \int_{D^k} l_i(x)l_j(x)\,\mathrm{d}x$ - the mass matrix

$S_{ij}^k = \int_{D^k} l_i(x)\frac{\mathrm{d}l_j(x)}{\mathrm{d}x}\,\mathrm{d}x$ - the stiffness matrix

$\mathcal{E}_{ij} = \begin{cases} 1 & : \text{ upper left corner and lower right corner} \\ 0 & : \text{ otherwise}\end{cases}$

$\left(\boldsymbol{u}_h^k\right)_j = u_h\left(x_j^k,t\right)$ - the vector of unknowns

$\left(\boldsymbol{f}_h^k\right)_j$ - the physical flux vector

$\boldsymbol{f}_*^k = \left(\boldsymbol{n}_L f^*\left(x_L^k\right), \boldsymbol{n}_R f^*\left(x_R^k\right)\right)^T$ - numerical flux at boundaries

$\boldsymbol{n}_L, \boldsymbol{n}_R$ - unit normal at the left and right element boundary respectively

This matrix equation is the semi-discrete form of the PDE. We will discuss all the local operatros in more detail later.

When choosing the basis of the space V_h^k we have many options. But for the mass matrix to be well conditioned we will choose the Legendre polynomials. [1, pg. 45]

2.2.1. Legendre polynomials

Legendre polynomials are a complete set of orthogonal polynomials defined on $\langle -1, 1\rangle$

An easy way to calculate the polynomials is through the three term reccurence using the Bonnet's formula [cite]

$$P_n(x) = \frac{(2n+1)xP_{n-1}(x) - nP_{n-2}(x)}{n+1}$$

To start the reccurence we need the first two terms. They are given as

$$P_0(x) = 1, \quad P_1(x) = x$$

Now to obtain the normalized (in the L^2 norm) Legendre polynomials $\tilde{P}_n(x)$ we multiply each polynomial by an appropriate coefficient

$$\tilde{P}_n(x) = \sqrt{\frac{2n+1}{2}}P_n(x)$$

To obtain a derivative of the Legendre polynomial we use the formula [cite]

$$P_n'(x) = \frac{n(P_{n-1}(x) - xP_n(x))}{1-x^2}$$

Notice that the formula doesn't hold for $x = \pm 1$, for that we use a the property

$$P_n'(1) = \frac{n(n+1)}{2}$$

and

$$P_n'(-x) = (-1)^{2n-1}P_n'(x)$$

2.3. Nodal representation

Let's now discuss the nodal representation in greater detail. We define $\hat{\boldsymbol{u}}_n$ such that the approximation is interpolatory i.e.

$$u(x_i) = \sum \hat{u}_n \psi_n(x_i),$$

where x_i represents distinct grid nodes. We can now write

$$\boldsymbol{u} = \mathcal{V}\hat{\boldsymbol{u}},$$

where $\mathcal{V}_{ij} = \psi_j(x_i)$ is the Vandermonde matrix and $u_i = u(x_i)$. This matrix connects the modes $\hat{u}$ and nodal values $u(x_i)$. We would like the matrix to be well conditioned. We already chose the basis polynomials, so only the nodes x_i are left, to define the Vandermonde matrix. The node's that acomplish this are the solution of

$$(1-x^2)\frac{\mathrm{d}}{\mathrm{d}x}P_N(x) = 0$$

these are known as the Legendre-Gauss-Lobatto nodes. The LGL nodes will be noted by the greek letter ξ. There are $N+1$ LGL nodes for solution approximation of order N. [1, pg. 47]

2.4. Local operators

Up till now we were developing a sensible local representation of the approximate solution. We should now discuss the various local operators in the nodal formulation of DGM.

2.4.1. Mass matrix

The local mass matrix M^k (on k-th element) is given as

$$M^k_{ij} = \int_{x^k_l}^{x^k_r} l^k_i(x) l^k_j(x)\,\mathrm{d}x = \frac{h_k}{2}\int_{-1}^{1} l_i(r) l_j(r)\,\mathrm{d}r = \frac{h_k}{2} M_{ij},$$

where the coefficient $\frac{h_k}{2}$ is a Jacobian coming from the affine transformation and M is the mass matrix defined on the reference element.

We know that

$$l_i(r) = \mathcal{W}_{in}\varphi_n(r),$$

where $\mathcal{W} = \left(\mathcal{V}^T\right)^{-1}$ From which we get

$$M_{ij} = \int_{-1}^{1} \mathcal{W}_{in}\varphi_n(r)\mathcal{W}_{jm}\varphi_m(r)\,\mathrm{d}r = \mathcal{W}_{im}\mathcal{W}_{jm}(\varphi_n, \varphi_m)_{L^2} =$$
$$= \mathcal{W}_{im}\mathcal{W}_{jm}$$

Thus

$$M^k = \frac{h_k}{2} M = \frac{h_k}{2}\left(\mathcal{V}\mathcal{V}^T\right)^{-1}$$

[1, pg. 51-52]

2.4.2. Stiffness matrix

The local stiffness matrix is given as

$$S^k_{ij} = \int_{x^k_l}^{x^k_r} l^k_i(x)\frac{\mathrm{d}}{\mathrm{d}x} l^k_j(x)\,\mathrm{d}x = \int_{-1}^{1} l_i(r)\frac{\mathrm{d}}{\mathrm{d}r} l_j(r)\,\mathrm{d}r = S_{ij}$$

To simply computate this we define the differentiation matrix.

$$D_{ij} = \frac{\mathrm{d}}{\mathrm{d}r} l_j \,|_{r_i}$$

Consider the product

$$(MD)_{ij} = M_{in}D_{nj} = \sum_n \int_{-1}^{1} l_i(r) l_n(r) \frac{\mathrm{d}}{\mathrm{d}r} l_j \,|_{r_n} = \int_{-1}^{1} l_i(r) \sum_n \frac{\mathrm{d}l_j}{\mathrm{d}r} \,|_{r_n} \, l_n(r)\,\mathrm{d}r =$$
$$= \int_{-1}^{1} l_i(r)\frac{\mathrm{d}l_j}{\mathrm{d}r}\,\mathrm{d}r = S_{ij}$$
$$MD = S$$

The entries of the differentiation matrix can be found directly

$$D = \mathcal{V}_r \mathcal{V}^{-1},$$

where $\mathcal{V}_r$ is the Vandermonde matrix assembled from differentiated Legendre polynomials.

$$(\mathcal{V}_r)_{ij} = \psi_j'(\xi_i)$$

But in the weak formulation S^T is involved, for that we use the identity

$$S^T = D^T M$$

To recover the semi-discrete formulation we need to multiply this by the mass matrix from the left

$$M^{-1} S^T = M^{-1} D^T M = \mathcal{V}\mathcal{V}_r^T M = D_w$$

We denote this matrix D_w, differentiation matrix for the weak form. [1, pg. 52-54]

2.4.3. Surface integral

This operator is responsible of extracting the surface terms of the form

$$\int_{\partial D^k} \hat{\boldsymbol{n}} f^* l_i(r) \,\mathrm{d}x = (\hat{\boldsymbol{n}} f^*) \mid_{x_R} \boldsymbol{e}_n + (\hat{\boldsymbol{n}} f^*) \mid_{x_L} \boldsymbol{e}_1,$$

where $\boldsymbol{e}_i$ is a zero vector with 1 at index i. We can rewrite this using a $N_p \times 2$ matrix $\mathcal{E}$ that is zero except for upper right corener that is 1 and the lower left corner that is -1 like so

$$\int_{\partial D^k} \hat{\boldsymbol{n}} f^* l_i(r) \,\mathrm{d}x = \mathcal{E} \cdot (f_R^*, f_L^*)^T$$

Again, to recover the semi-discrete form we multiply by the inverse mass matrix from the left. We call this matrix the L.

$$L = M^{-1}\mathcal{E} = \mathcal{V}\mathcal{V}^T\mathcal{E}$$

[1, pg. 56]

2.5. Time discretization

Two explicit RK4 methods were implemented to integrate the semidiscrete system

$$\frac{\mathrm{d}u_h}{\mathrm{d}t} = \mathcal{L}_h(u_h, t)$$

2.5.1. ERK

General explicit four stage Runge-Kutta method. [1, pg. 63]

$$\begin{aligned}
k_1 &= \mathcal{L}_h(u_h^n, t^n) \\
k_2 &= \mathcal{L}_h\left(u_h^n + \frac{1}{2}\Delta t k_1, t^n + \Delta t\right) \\
k_3 &= \mathcal{L}_h\left(u_h^n + \frac{1}{2}\Delta t k_2, t^n + \Delta t\right) \\
k_4 &= \mathcal{L}_h\left(u_h^n + \frac{1}{2}\Delta t k_3, t^n + \Delta t\right) \\
u_h^{n+1} &= u_h^n + \frac{1}{6}\Delta t(k_1 + k_2 + k_3 + k_4)
\end{aligned}$$

2.5.2. SSPRK

Strong stability preserving Runge-Kutta fourth order method. [2]

$$
\begin{aligned}
k_1 &= u^n + \frac{1}{2}\Delta t \mathcal{L}_h(u^n, t_n) \\
k_2 &= k_1 + \frac{1}{2}\Delta t \mathcal{L}_h\Big(k_1, t_n + \frac{1}{2}\Delta t\Big) \\
k_3 &= \frac{2}{3}u^n + \frac{1}{2}k_2 + \frac{1}{6}\mathcal{L}_h(k_2, t^n + \Delta t) \\
u_h^{n+1} &= k_3 + \frac{1}{2}\Delta t \mathcal{L}_h\Big(k_3, t^n + \frac{1}{2}\Delta t\Big)
\end{aligned}
$$

2.6. Example linear problem

We are given the following problem

$$\partial_t u + \partial_x u = 0, \quad x \in \langle -1, 1\rangle$$

with periodic boundary conditions.

We assume the solution can be approximated as

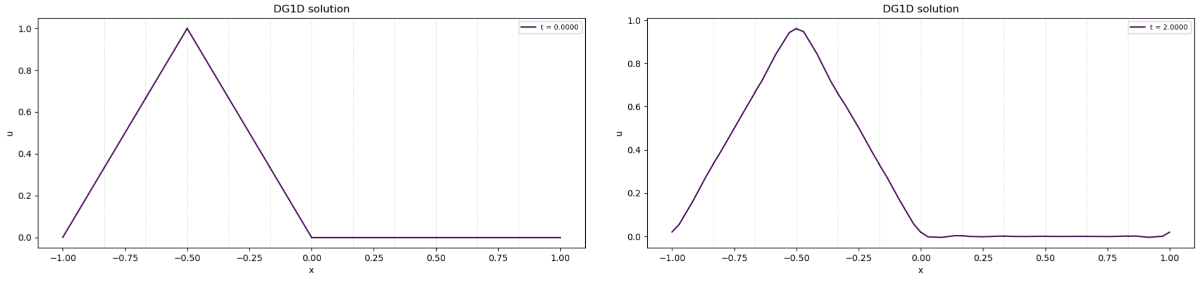
$$u_h^k = \sum_i u_h^k(x_i^k, t) l_i^k(x).$$

And that the direct sum of these solutions is an approximation to the global solution. This yields the local semidiscrete scheme

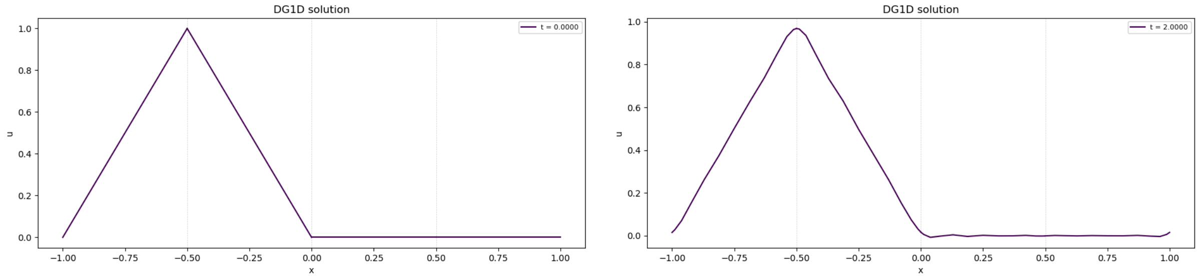
$$
\begin{aligned}
M^k \frac{\mathrm{d}\boldsymbol{u}_h^k}{\mathrm{d}t} - S^T \boldsymbol{u}_h^k &= -[l^k(x) f^*(u))]_{x_l^k}^{x_r^k} = -\mathcal{E}(f^*(u), f^*(u))^T \\
J^k M \frac{\mathrm{d}\boldsymbol{u}_h^k}{\mathrm{d}t} - S^T \boldsymbol{u}_h^k &= -\mathcal{E}(f^*(u), f^*(u))^T \\
\frac{\mathrm{d}\boldsymbol{u}_h^k}{\mathrm{d}t} &= \frac{1}{J^k}\Big(M^{-1} S^T \boldsymbol{u}_h^k - M^{-1}\mathcal{E}(f_R^*, f_L^*)^T\Big) \\
\frac{\mathrm{d}\boldsymbol{u}_h^k}{\mathrm{d}t} &= \frac{1}{J^k}\Big(D_w \boldsymbol{u}_h^k - L(f_R^*, f_L^*)^T\Big)
\end{aligned}
$$

We have defined all of these operators beforehand in Section 2.4.

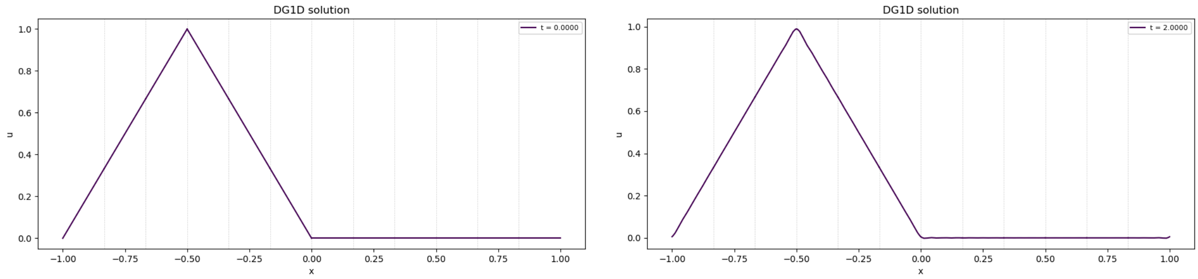
Now let's test our solver on multiple initial conditions with different number of elements (K) and polynomial order (N). First we will test non-smooth initial conditions. On the left are the initial conditions and on the right is the solution after one period. (Keep in mind we are using periodic boundary conditions)



Cone with $K = 12$, $N = 4$

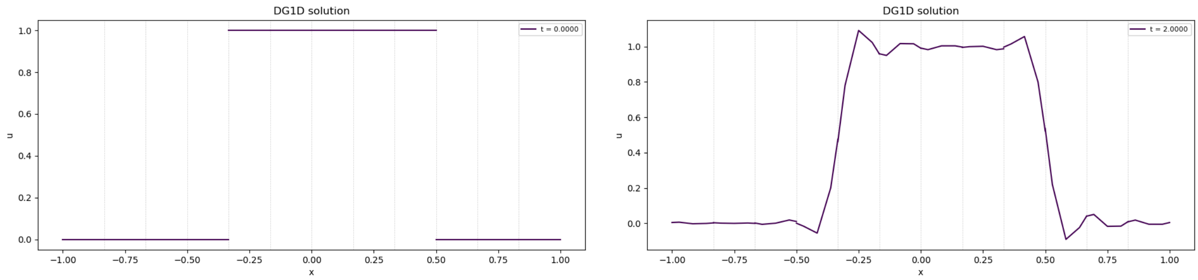


Cone with $K = 4$, $N = 12$

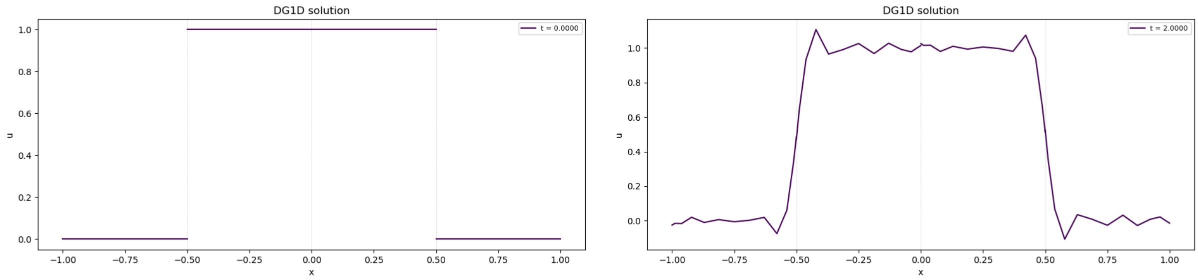


Cone with $K = 12$, $N = 12$

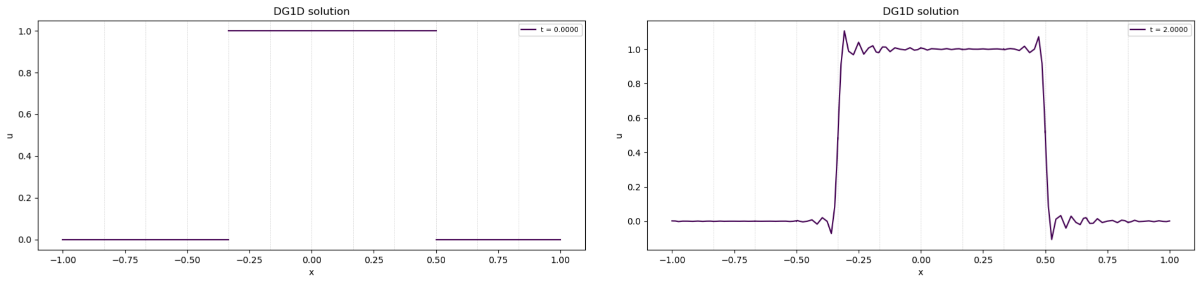
Now let us test a discontinuous initial condition in the form of a step function.



Step signal with $K = 12$, $N = 4$



Step signal with $K = 4$, $N = 12$



Step signal with $K = 12$, $N = 12$

The last figure shows a beautiful example of Gibb's phenomenon, that stems from trying to approximate discontinuous function using a polynomial. There are a few approaches how to battle these oscillations. One is using a filter on the solution after each timestep and the other is using a suitable solution reconstruction.

Bibliography

[1] J. S. Hesthaven and T. Warburton, *Nodal discontinuous Galerkin methods: algorithms, analysis, and applications.* Springer, 2008.

[2] D. R. Durran, *Numerical Methods for Fluid Dynamics.* Springer, 2010.